

Contents

1	CPU Scheduling	2
1.1	Scheduling dei Processi	2
1.1.1	Concetti Generali	2
1.1.2	Compiti del Dispatcher	2
1.1.3	Parametri Generali	2
1.1.4	Obbiettivi	3
1.1.5	Criteri per CPU Scheduling	3
1.1.6	CPU-burst e I/O-burst	4
1.1.7	Scheduling Preemptive e Non-preemptive	4
1.2	Algoritmi di Scheduling Processi	5
1.2.1	FCFS - First Come First Served	5
1.2.2	SJF - Shortest Job First	5
1.2.3	SRTF - Shortest Remaining Time First	6
1.2.4	Caratteristiche SJF / SRTF	6
1.2.5	Stima durata CPU burst	6
1.2.6	HRRF - Highest Response Ratio First	7
1.2.7	Scheduling con Priorità	7
1.2.8	Round Robin (circolare)	8
1.2.9	Durata quanto di tempo	8
1.2.10	Scheduling con Code Multiple	8
1.2.11	Scheduling con Code Multiple -Retroazione	9
1.2.12	Comparazione tra algoritmi	10
1.2.13	Fair Share Scheduling	10
1.3	Scheduling dei Thread	10
1.3.1	Kernel-Thread e User-Thread	10
1.4	Scheduling per Sistemi Multiprocessore	11
1.4.1	Scheduling in Sistemi Multiprocessore	11
1.4.2	Bilanciamento del Carico	12
1.4.3	Sistemi Eterogenei	12
1.5	Scheduling Real-time	12
1.5.1	Sistemi Real-Time	12
1.6	Scheduling in UNIX e Linux	13
1.6.1	Scheduling di UNIX Tradizionale	13
1.6.2	Scheduling in O(1) di Linux	13

lololol

1 CPU Scheduling

1.1 Scheduling dei Processi

1.1.1 Concetti Generali

In generale risolvere un problema di scheduling consiste nel **mappare** un certo numero di **compiti** su un (diverso) numero di **esecutori** in modo da realizzare obbiettivi globali del sistema rispettando **vincoli** prefissati.

I principali **eventi** che possono causare uno **switch di contesto** (e che fanno quindi entrare in gioco lo scheduler) sono:

- interruzione relativa a timer (termine del quanto di tempo)
- interruzione dovuta a dispositivi I/O
- invocazione di system call (es. `exit()`)
- gestione di segnali

1.1.2 Compiti del Dispatcher

Lo scheduler sceglie il processo da eseguire, successivamente il dispatcher:

- effettua il context switch
- inizializza program counter alla prossima istruzione del programma da (continuare a) eseguire
- effettua il passaggio alla modalità utente

Il tempo per fare ciò viene detto latenza di dispatch.

1.1.3 Parametri Generali

- numero di processori (1+), omogeneità/disomogeneità dei processori (in tipo, velocità, ISA...)
- staticità/dinamicità insieme dei task
- on-line e off-line scheduling
- tempi di esecuzione noti a priori o sconosciuti
- presenza o meno di preemption (prelazione)
- presenza o meno di precedenza tra task
- presenza o meno di priorità

- imposizione o meno di scadenze (deadline) su inizio/terminazione task
- presenza o meno di **obbiettivo** da ottimizzare (tempo di risposta, tempo di completamento, produttività...)

1.1.4 Obbiettivi

Obbiettivi diversi, spesso antitetici, sono ottimizzazione di:

- lunghezza dello schedule
- **tempo di risposta** massimo
- **tempo di completamento** medio
- tempo di risposta medio
- tempo di attesa
- numero minimo di CPU utilizzate
- produttività (num. task/tempo $\rightarrow$ **throughput**)
- utilizzo della CPU
- massimo ritardo rispetto a eventuali deadline

1.1.5 Criteri per CPU Scheduling

Dal punto di vista **utente**:

- minimizzare tempo risposta
- minimizzare tempo completamento
- rispettare deadline

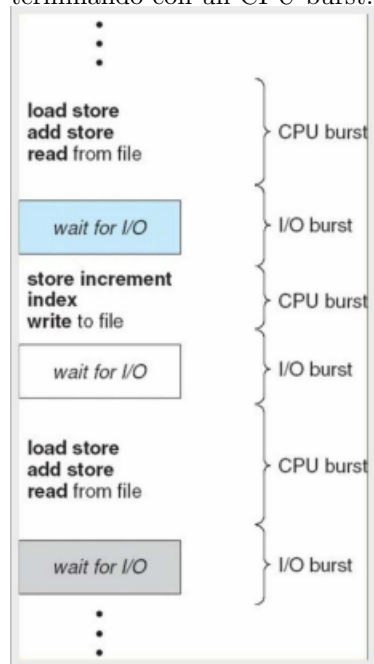
Dal punto di vista **sistema**:

- massimizzare utilizzo CPU
- garantire **fairness** (problema della **starvation**)
- massimizzare throughput
- riflettere le priorità

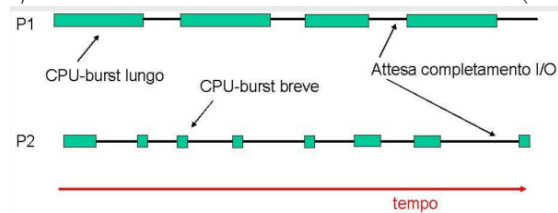
Sia scheduler che dispatcher rappresentano bottleneck per il S.O.

1.1.6 CPU-burst e I/O-burst

Il comportamento tipico di un processo consiste nell'alternare sequenze di operazioni di elaborazione (CPU-burst) a sequenze di operazioni di I/O (I/O burst), terminando con un CPU-burst.



Processi CPU-bound avranno pochi CPU-burst lunghi (P1), mentre processi I/O-bound avranno molti CPU-burst brevi (P2):



1.1.7 Scheduling Preemptive e Non-preemptive

Lo schema di scheduling può essere con o senza **diritto di prelazione**:

- **Nonpreemptive scheduling:** processo running continua a detenere la CPU fino a quando termina o la cede
- **Preemptive scheduling:** processo può essere interrotto (portato a ready) a seguito di un evento “esterno” (scade quanto di tempo, altro processo diventa ready...)

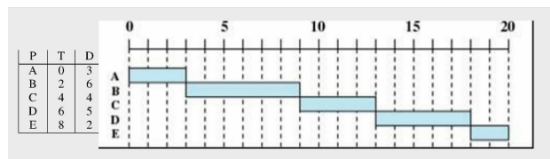
Nonpreemptive scheduling tipico dei sistemi batch, preemptive scheduling evita

che processi “meno importanti” ostacolano quelli critici (comportando però un overhead di gestione)

1.2 Algoritmi di Scheduling Processi

1.2.1 FCFS - First Come First Served

- Si seleziona il processo che è nella ready-queue da più tempo (processi serviti nell'ordine in cui entrano nella ready-queue)
- Nonpreemptive

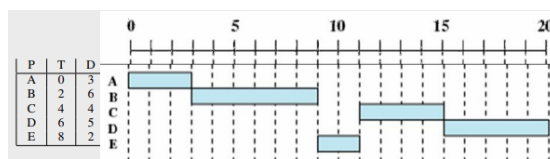


- Vantaggi
 - Semplice da implementare, non comporta overhead di gestione
 - Non ci può essere starvation in quanto tutti i processi ready, prima o poi, ottengono uso CPU
- Svantaggi
 - Tempo d'attesa medio molto legato all'ordine di arrivo dei burst
 - Tempo di risposta di un processo può essere molto alto anche per processi di breve durata
 - Un processo CPU-bound tende a monopolizzare il processore a discapito dei processi I/O-bound
 1. Processi I/O-bound terminano burst di I/O e si accodano in ready
 2. CPU-bound rilascia la CPU per fare I/O
 3. Tutti gli I/O-bound terminano il burst di CPU per fare I/O
 4. Il CPU-bound termina l'I/O e torna ready ("prima" degli I/O-bound) e tutto si ripete

Si tende quindi a sottoutilizzare sia device che CPU (convoy effect)

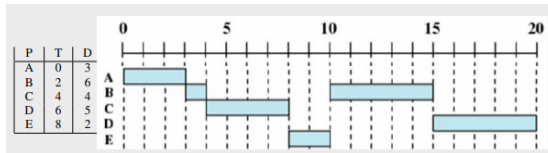
1.2.2 SJF - Shortest Job First

- Si seleziona il processo che avrà il prossimo CPU-burst più breve
- Nonpreemptive



1.2.3 SRTF - Shortest Remaining Time First

- Si seleziona il processo che avrà il prossimo CPU-burst più breve
- Se P1 è running e in ready-queue giunge P2 con durata inferiore al tempo restante di P1, allora P2 sottrae la CPU a P1
- Preemptive



1.2.4 Caratteristiche SJF / SRTF

- Processi con CPU-burst più breve sopravanzano gli altri
- SJF rappresenta lo scheduling ottimale: realizza il tempo di attesa medio migliore
- Può esserci starvation: i processi con CPU-burst lunghi tendono a restare molto tempo in ready-queue, non garantito ricevano l'uso della CPU in un tempo limitato
- Richiedono di prevedere la durata dei CPU-burst (non sempre possibile determinare queste durate)

1.2.5 Stima durata CPU burst

La previsione/stima della durata del prossimo CPU-burst di un processo può essere compiuta basandosi sui suoi CPU-burst del passato tramite media esponenziale:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

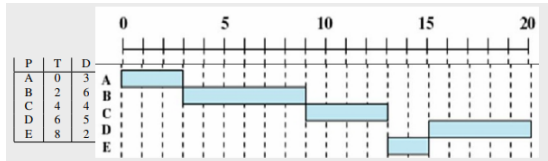
dove t_n è la durata dell'n-esimo CPU-burst cioè la storia recente, mentre τ_n rappresenta il contributo della storia remota. Sviluppando il termine n-esimo si ottiene:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\alpha\tau_{n-1} + (1 - \alpha)^2\alpha\tau_{n-2} + \dots + (1 - \alpha)^{n+1}\tau_0$$

con τ_0 e $0 \leq \alpha \leq 1$ costanti predefinite. In particolare α permette di modulare il peso della storia recente/remota.

1.2.6 HRRF - Highest Response Ratio First

- Si seleziona il processo che ha il **Response Ratio** R maggiore, dove $R = (t_a + t_s) / t_s$ con t_a il tempo che il processo ha speso in attesa della CPU e t_s la durata prevista del suo CPU-burst
- Nonpreemptive



- Processi con CPU-burst brevi sono favoriti ma si tiene conto dell'età del processo, processi con tempo di servizio t_s lungo guadagnano priorità man mano che invecchiano
- Meccanismo di **aging**: evita la starvation
- Come SJF e SRTF richiede stime su tempo di servizio richiesto da processi

1.2.7 Scheduling con Priorità

Viene assegnata una priorità ad ogni processo e CPU viene allocata a processo ready con priorità superiore. Nonpreemptive o preemptive (in questo caso un processo ready con priorità superiore sottrae la CPU ad un processo running di priorità inferiore)

- Gestione delle priorità
 - Priorità statiche
 - * priorità assegnata ad un processo resta costante
 - * facile implementazione
 - * minimo overhead di gestione
 - * no adattamento a cambiamenti del carico di sistema
 - Priorità dinamiche
 - * reattività rispetto a mutazioni del carico di sistema
 - * permettono di “seguire” evoluzione del comportamento dei processi
 - * maggior overhead per gestione
 - * aumento dinamico (solitamente) della priorità ai processi I/O-bound

- Priorità e Starvation Starvation = processo P, pur essendo ready, non ottiene mai uso della CPU.

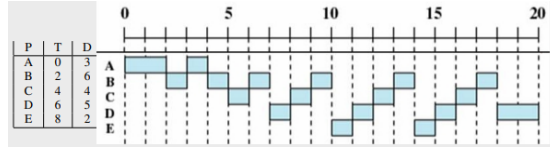
In una politica con priorità potrebbe verificarsi se venissero continuamente creati processi con priorità superiore a quella di P.

Per evitare starvation si usa una strategia di aging dei processi (essi tendono a migliorare la loro priorità con il passare del tempo)

1.2.8 Round Robin (circolare)

Come per FCFS si serve il processo che attende da più tempo

Preemptive usando timer: allo scadere del quale il processo running viene rimesso in ready-queue



Processi I/O-bound penalizzati (in generale hanno CPU-burst più brevi del quanto di tempo), burst termina prima della fine del quanto (vengono messi in waiting prima di aver sfruttato tutto tempo a disposizione)

Processi CPU-bound tendono a esaurire il quanto, quindi anche se mix tra CPU e I/O bound è adeguato in media i CPU-bound sfruttano (tendenzialmente) di più la CPU. Vantaggio: evita la starvation

1.2.9 Durata quanto di tempo

Durata di un quanto è critica per il comportamento del sistema:

- deve essere sensibilmente superiore al tempo necessario per gestire interruzioni / switch di contesto
- si rilevano buoni risultati se durata del quanto è maggiore della durata di almeno 80% di CPU-burst
- non può essere troppo lungo → penalizza processi I/O-bound (tende ad un FCFS)

1.2.10 Scheduling con Code Multiple

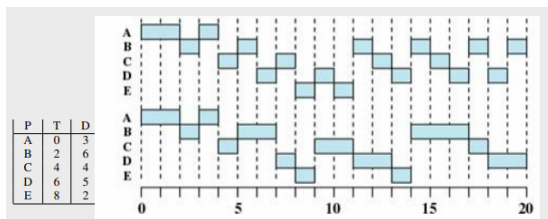
Più code di processi:

- processi partizionati in gruppi disgiunti secondo opportuni criteri (es. processi batch e processi interattivi)
- processi assegnati permanentemente alle code
- ogni coda gestita mediante algoritmo di scheduling più adatto a caratteristiche dei suoi processi
- si deve stabilire uno scheduling di code:
 - relazione di priorità tra code e strategia preemptive: processo di una coda viene interrotto appena un altro processo entra in una delle code con priorità maggiore
 - **time slicing** a livello delle code: ad ogni coda viene assegnata una percentuale di tempo totale di CPU

1.2.11 Scheduling con Code Multiple -Retroazione

Evoluzione delle code multiple:

- **Evita starvation**, permettendo ai processi di migrare tra code secondo criteri di **aging**.
- Solitamente **preemptive**.
- I processi che utilizzano a lungo la CPU vengono spostati in code con **priorità più bassa**.
- I processi **interattivi (I/O-bound)** tendono invece a ricevere **priorità più alta**.
- Un processo che resta troppo tempo in una coda a bassa priorità può essere **promosso** a una coda con priorità maggiore (per evitare starvation).
- **Overhead di gestione** viene bilanciato da una maggiore **sensibilità e reattività del sistema**.



Parametri da considerare:

- Numero code da utilizzare
- Algoritmi di scheduling per ogni coda.
- **Regole di migrazione** dei processi tra le code (es. un processo preempted può “retrocedere”, mentre l'**aging** lo fa salire di priorità).
- **Posizione iniziale** nuovi processi (es. si può scegliere di inserirli direttamente nella coda con priorità più alta).

1.2.12 Comparazione tra algoritmi

	Selection Function	Decision Mode	Throughput	Response Time	Overhead	Effect on Processes	Starvation
FCFS	$\max[w]$	Nonpreemptive	Not emphasized	May be high, especially if there is a large variance in process execution times	Minimum	Penalizes short processes; penalizes I/O bound processes	No
Round Robin	constant	Preemptive (at time quantum)	May be low if quantum is too small	Provides good response time for short processes	Minimum	Fair treatment	No
SPN	$\min[s]$	Nonpreemptive	High	Provides good response time for short processes	Can be high	Penalizes long processes	Possible
SRT	$\min[s - e]$	Preemptive (at arrival)	High	Provides good response time	Can be high	Penalizes long processes	Possible
HRRN	$\max\left(\frac{w + s}{s}\right)$	Nonpreemptive	High	Provides good response time	Can be high	Good balance	No
Feedback	(see text)	Preemptive (at time quantum)	Not emphasized	Not emphasized	Can be high	May favor I/O bound processes	Possible

1.2.13 Fair Share Scheduling

Processi **partizionati in gruppi** e scheduling avviene mirando a **equità di trattamento rispetto ai gruppi**:

- adatto a sistemi multi utente
- adatto in presenza di applicazioni multi-thread e scheduling viene effettuato sui thread
- può trattare diversamente processi di differenti tipologie di utente

1.3 Scheduling dei Thread

1.3.1 Kernel-Thread e User-Thread

- User Threads (UT)
 - Gestiti nello spazio utente (user space).
 - **Il kernel non ne è consapevole.**
 - Più veloci da creare e gestire.
 - **Problema:** se un thread effettua una system call bloccante, blocca tutti i thread del processo
 - **PROCESS-CONTENTION SCOPE**
- Kernel Threads (KT)
 - Gestiti dal kernel, nello spazio kernel.
 - Ogni thread è schedulato **indipendentemente**.
 - Più costosi da gestire in termini di performance.
 - **SYSTEM-CONTENTION SCOPE**

1.4 Scheduling per Sistemi Multiprocessore

1.4.1 Scheduling in Sistemi Multiprocessore

In linea di principio possono essere adattate le tecniche di scheduling per i casi single-processor anche in presenza di più unità di elaborazione.

Il carico deve però essere distribuito e **bilanciato**. **Tipologie di multielaborazione**

- **Asimmetrica:** attività di sistema (scheduling, gestione, interruzioni...) affidate tutte a un unico processore **master server**
 - codice degli utenti eseguito dagli altri processori
 - **difetto:** master server è un potenziale bottleneck del sistema
- **Simmetrica:** ogni processore ha uno scheduler che attinge alla ready-queue che può essere
 - di sistema: possibili race condition se più processori scelgono stesso task (serve mutua esclusione all'interno di ready-queue → possibile bottleneck)
 - per-processor: serve un meccanismo per distribuzione/bilanciamento carico tra ready-queue

Scheduling (Multi)processori Multicore

- Processori multicore presentano più core in stesso chip fisico
- S.O. vede ogni core come un processore logico separato
- Ogni core esegue (solitamente) più thread hardware: **chip multithreading, hyper-threading**
- Presenza di multithreading a livello di core comporta doppio livello di scheduling:
 - scheduling (hardware) di thread hardware sui core della CPU multicore
 - scheduling (software) di thread/processi/task sulle CPU (lo scheduling studiato finora)

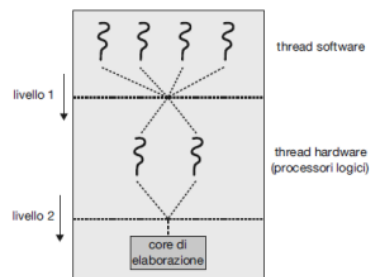


Figura 5.15 Due livelli di scheduling.

1.4.2 Bilanciamento del Carico

Con più unità di elaborazione va massimizzato distribuendo equamente il carico tra i core del sistema, si utilizzano principalmente due tecniche base (non esclusive):

- Push migration: un processo dedicato controlla periodicamente carico di lavoro assegnato a predecessori → sottrae task da CPU overloaded e li riassegna a CPU meno cariche
- Pull migration: ogni processore, quando inattivo, preleva un task da uno dei processori overloaded

1.4.3 Sistemi Eterogenei

Presenza di diversi processori con caratteristiche non omogenee:

- diverso tipo (processori grafici / dedicati a task specifici)
- diverso set di istruzioni
- diversa potenza di calcolo / consumo energetico

Utile in caso si debba dare maggiore importanza a consumo energetico (sistemi embedded...)

1.5 Scheduling Real-time

1.5.1 Sistemi Real-Time

- Utilizzati in dispositivi di largo consumo (sistemi embedded, player, robotica, domotica)
- Funzionalità devono rispettare vincoli temporali (scadenze, durate, tempi di reazione...)
- Obiettivo primario rispetto di deadline e vincoli ai tempi di reazione
- Necessitano algoritmi di scheduling specifici

Tipologie di Sistemi Real-Time:

- Real-time soft: ai task critici viene concessa precedenza (no violazioni deadline però)
- Real-time hard: task devono necessariamente rispettare deadline

Solitamente sono event-driven (all'accadere di un evento il sistema reagisce nel minor tempo possibile ed esegue il task richiesto → tempo di reazione detto latenza dell'evento)

- **latenza di interruzione:** tempo trascorso tra la ricezione dell'interruzione e l'inizio della procedura di gestione
- **latenza di dispatch:** tempo impiegato dal dispatcher per eseguire lo switch di contesto

Scheduling Sistemi Real-Time

- **Priorità:** scheduling con priorità con prelazione visto in precedenza o varianti per garantire deadline hard e gestire task periodici
- **Priorità proporzionale a frequenza:** come precedente ma con priorità assegnate staticamente in base a frequenza del task periodico
- **Earliest Deadline First:** priorità assegnate dinamicamente (prima il task che ha scadenza più vicina, non richiede periodicità)
- **A quote proporzionali:** assegnata una percentuale del tempo di CPU a ogni applicazione, a ogni richiesta di utilizzo CPU si valuta se ammetterla o meno

1.6 Scheduling in UNIX e Linux

1.6.1 Scheduling di UNIX Tradizionale

- Utilizza **code multiple con priorità dinamiche** e algoritmo **round-robin** per ogni coda.
- **Preemption** dopo 1 secondo di utilizzo continuato della CPU.
- Ogni processo ha una **priorità di base** che lo colloca in una banda specifica (es. Swapper, User processes) per ottimizzare l'uso dei dispositivi I/O.
- La priorità viene modificata in base all'utilizzo della CPU e alle preferenze dell'utente (nice), ma rimane nella stessa banda.
- Le priorità sono ricalcolate ogni secondo.

1.6.2 Scheduling in O(1) di Linux

- Con **preemption**.
- Basato su **priorità numeriche** con due gamme:
 - Priorità alta (es. 0) → quanto lungo (200 ms).
 - Priorità bassa (es. 140) → quanto breve (10 ms).

Gestione dei Task

- **Task ready:** classificati come **active** (se hanno tempo residuo) o **expired** (se hanno esaurito il quanto).

- Il task **running** diventa **expired** quando consuma il suo quanto e non può essere rieseguito finché ci sono task active.
- Due array gestiscono task active e expired. La CPU è assegnata al task active con priorità massima.
- Gli array si scambiano i ruoli quando tutti i task **active** diventano **expired**.